

Metronome: Bound the Cache, Keep the Beat for Real-Time Interaction Model Serving

Jiaying Meng
Independent Researcher

Bojie Li
Pine AI

Abstract

Real-time interaction models — Moshi, MiniCPM-o, Qwen-Omni — turn serving into a *periodic real-time task*: on every frame a session ingests streaming audio and must respond by a recurring wall-clock deadline, while its KV cache grows monotonically and stays pinned for the whole conversation. This regime hides a dangerous failure mode. On a real full-duplex stack, sustained load does not degrade serving gracefully: it **falls off a cliff**, jumping in one step from milliseconds per frame to a stalled engine when accumulated session state exhausts the KV pool. The collapse is *metastable* — identical five-minute runs collapse or survive on run-to-run variance — and *silent*: latency and deadline-miss metrics read healthy throughout.

We show one move restores both stability and observability: **bound each session’s resident state, and latency starts telling the truth**. Metronome’s in-engine KV window eliminates the collapse (0/20 vs. 14/20 runs across two batches) and turns per-frame latency into a monotone load signal, on which an online admission controller discovers the schedulable concurrency; without the window, the identical controller over-admits into the wall. A first-order model predicts the collapse time within a few percent on the headline model, and a quality probe validates the bound’s design by ablation: the window alone is quality-free in turn-based decoding, and its few pinned attention-sink tokens are what keep free-running generation healthy. Everything is measured end-to-end on real audio, across four interaction models on one GPU.

Code: <https://github.com/19PINE-AI/metronome>

Website: <https://01.me/research/metronome>

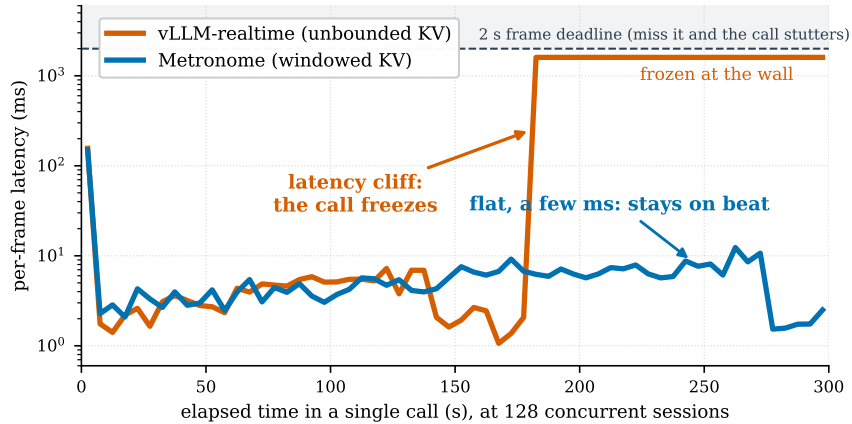


Figure 1. Existing serving falls off a latency cliff; Metronome stays on beat. One five-minute call at 128 sessions (Qwen3-Omni-30B): unbounded resident KV jumps in one step from a few ms to a ~1.6 s ceiling where the stalled engine stops producing tokens; windowed KV stays flat for the whole call.

1 Introduction

A new class of models has quietly created a serving regime of its own. Kyutai Moshi [Défossez et al., 2024], MiniCPM-o-4.5 [OpenBMB, 2026], and the Qwen-Omni family [Chu et al., 2024, Xu et al., 2025] listen to streaming audio and respond continuously, with no turn boundary, and a growing line of streaming speech LLMs follows the same shape [Xie and Wu, 2024, Fang et al., 2024]. Frontier labs are converging on it: Thinking Machines Lab’s *interaction models* interleave 200 ms micro-turns of multimodal input and output on a persistent GPU sequence [Thinking Machines Lab, 2026]. At the serving layer these models are not chatbots. A chatbot request is *ephemeral*: a prompt arrives, tokens are generated, the request and its KV cache are freed. An interaction *session* is a *periodic real-time task*: on every frame — every 80 ms for Moshi, every 1–2 s for the omni models — it ingests a new audio chunk as a small prefill and decodes a short response, against a recurring wall-clock deadline, for a conversation that runs for minutes (Figure 3).

This regime sits at an unstudied intersection. LLM serving assumes requests are ephemeral: engines maximize aggregate throughput and reclaim or swap KV between requests [Yu et al., 2022, Kwon et al., 2023, Gujarati et al., 2020], parking whatever state must persist out of GPU memory during the idle gaps between turns. Classical real-time scheduling assumes periodic tasks have *bounded* state [Liu and Layland, 1973]. An interaction session violates both: its deadline recurs frame after frame, and its per-session KV cache grows monotonically and stays pinned — it never leaves on its own. It also has no idle gap in which to swap that state out or recompute it, the escape hatches turn-based serving relies on (§2), so its KV must stay *resident*. The state-of-the-art engines have grown exactly this mechanism — vLLM’s resumable requests [vLLM Team, 2026] and SGLang’s streaming sessions [Zheng et al., 2024, Thinking Machines Lab, 2026] keep a session’s KV live on the GPU across frames — but neither addresses the unbounded-state half.

We show that this combination fails in a characteristic and dangerous way. Driving the unmodified resumable-KV path with real audio through a real full-duplex stack — WebSocket clients, a Go gateway, a GPU worker — we find that sustained load does not degrade interaction serving gracefully. *It falls off a cliff* (Figure 1): per-frame latency holds at a few milliseconds and then, in a single step, pins at a ceiling where the stalled engine stops producing tokens. Three properties define the failure (§3). It is a **memory** event: per-frame compute stays cheap to the last tick, but each session’s KV grows every frame until the engine’s block pool saturates and the scheduler stalls. It is **metastable**: identical five-minute runs collapse or survive on run-to-run variance in how fast the pool fills. And it is **silent**: the stalled engine returns empty frames on time, so latency and deadline-miss metrics read healthy through the collapse — the call simply goes quiet.

A cliff is not merely a performance bug; it is a *control* bug. Admission, autoscaling, and load shedding all act on feedback, and a signal that stays flat until the instant of collapse carries no information to act on. Graceful degradation is therefore not a nicety; it is the *precondition* for every overload defense one would build. The central finding of this paper is that one move restores both stability and observability at once: **bound each session’s resident state, and latency starts telling the truth**. Capping the resident context turns the cliff into a slope: per-frame latency rises smoothly and monotonically with load, which keeps every session on beat and gives a feedback controller a signal it can trust.

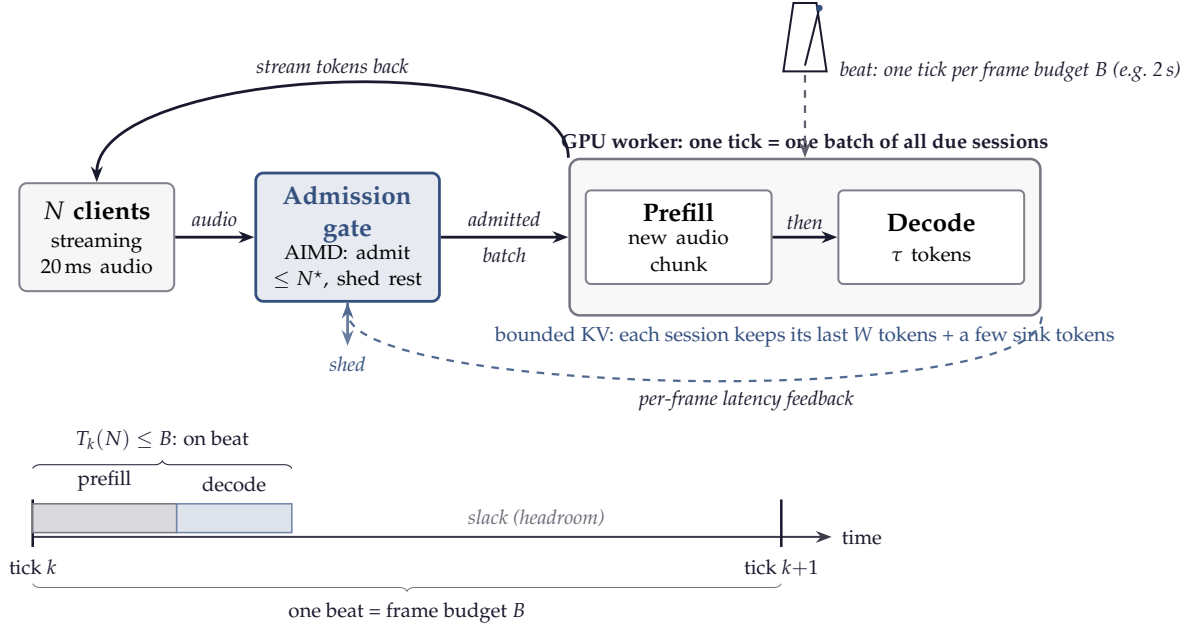


Figure 2. Metronome keeps the beat. A metronome *tick* fires once per frame budget B . On each tick the system serves one *batch* of all due sessions — prefill each session’s new audio chunk, then decode a few tokens (continuous batching) — and must finish within B ; the timeline at the bottom shows one tick’s anatomy, with the slack as schedulability headroom. Two interdependent mechanisms keep the beat steady: (a) an *admission gate* (AIMD, in the gateway) admits up to the schedulable concurrency N^* and sheds the surplus, driven by the per-frame latency it measures; (b) an *in-engine windowed KV* (in the worker) retains only each session’s last W tokens plus a few pinned attention-sink tokens, which bounds per-frame work, keeps free-running generation healthy (§5.4), and, by turning the latency cliff into a slope, makes that feedback signal trustworthy.

Metronome. We realize this principle as Metronome (Figure 2) and keep the mechanism deliberately minimal, because the contribution is knowing *what* to bound and *why*, not machinery. An *in-engine windowed KV* activates the engine’s dormant sliding-window path so that each session attends over — and, crucially, retains — only its most recent W tokens plus a few pinned attention-sink tokens that keep a full-attention backbone generating cleanly (§4.1). On the latency signal this restores, an online AIMD *admission controller* discovers the schedulable concurrency N^* and sheds the surplus cleanly (§4.2). The dependency between the two is demonstrated, not asserted: the identical controller converges on bounded state and over-admits straight into the wall without it (§5.3). A first-order model of pool fill makes the collapse predictable, not just observable (§3.1).

Contributions.

1. **Characterization.** Real-time interaction serving is periodic serving with unbounded per-session state. On a real full-duplex stack, we show that this combination fails by a *memory-triggered, metastable, silent* latency cliff, and we give a validated first-order model that predicts when it strikes and explains its apparent randomness (Sections 3, 3.1).
2. **Principle.** Bounding each session’s resident KV removes the cliff *and* restores a monotone latency signal. One mechanism buys both stability and observability, and the second is what any overload control requires (§4).

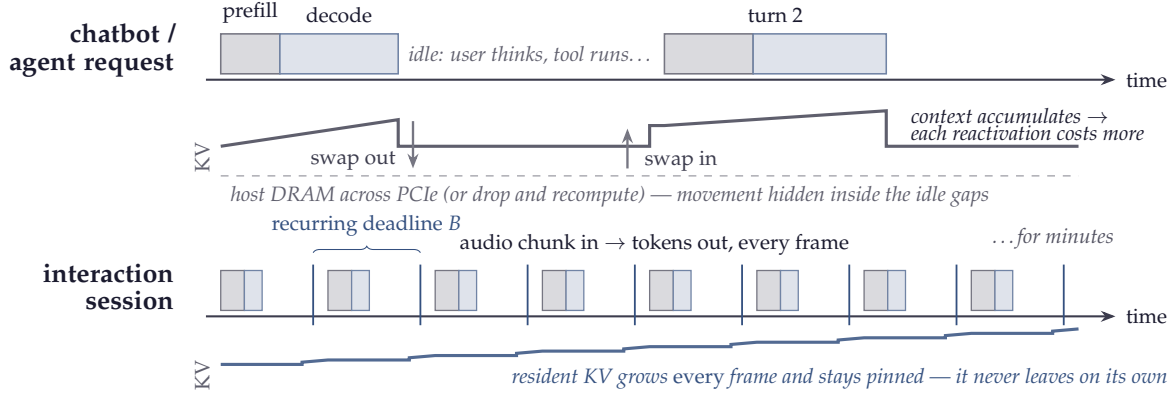


Figure 3. The workload engines were not built for. (a) A chatbot or agent request is *turn-based*: bursts of prefill and decode separated by idle gaps. Its context accumulates, so between turns the engine parks the KV out of GPU memory — swapped to host, or dropped and recomputed — and pays a *reactivation toll* that grows with the conversation; the idle gaps hide the movement, and a slow reactivation delays only one reply. **(b)** An interaction session is *persistent and periodic*: a small prefill and a short decode against a recurring wall-clock deadline B on every frame, with no idle gap, for the whole conversation — its KV must stay resident (§2), and resident it grows monotonically, pinned for the session’s life. Periodic deadlines plus unbounded per-session state is the combination this paper studies.

3. **Demonstration.** A minimal in-engine bound — sliding window plus pinned attention sinks — and a deadline-aware admission controller that demonstrably depends on it, measured end-to-end on real audio across four interaction models, with each half’s role in generation quality validated by ablation (§5).

2 Interaction Sessions Are Periodic Real-Time Tasks

The task model. A session presents a new audio chunk once per frame; we take the frame period equal to the frame budget B . At frame k the engine must encode and prefill the new chunk, then decode up to τ output tokens, attending over the session’s accumulated context. With N concurrent sessions the engine does this for all due sessions each tick under continuous batching [Yu et al., 2022]. The session is schedulable iff the per-frame wall time satisfies $T_k(N) \leq B$ for every k — a recurring-deadline condition in the classical real-time sense [Liu and Layland, 1973]. Two properties separate this from chatbot serving (Figure 3). The deadline *recurs*, so tail behavior compounds over thousands of frames rather than being amortized away. And the per-session KV cache is *pinned*: it cannot be reclaimed mid-session the way throughput-oriented swapping assumes, and left to itself it grows with every frame for the life of the conversation.

Recompute, swap, or stay resident. Session state that must persist has three places to live, each taxing a different resource. It can be *recomputed* — freed, then re-encoded from recent context on reactivation, echoing sliding-window and streaming attention [Beltagy et al., 2020, Jiang et al., 2023, Xiao et al., 2024] — a compute toll that grows with the context. It can be *swapped* to host memory and moved back over the interconnect when next served, as throughput-oriented engines do between requests [Kwon et al., 2023, Sheng et al., 2023] — a bandwidth toll that likewise grows. Or it can stay *resident* in GPU memory: no reactivation

toll, but the memory is held. Turn-based serving can afford the first two because a chatbot’s idle gaps — user think time, tool calls — hide the movement. An interaction session has no lull: *every* session is due on *every* frame, so a recompute or swap toll recurs against the frame budget and grows with session age — at our operating points, swapping each due session’s state out and back every frame would move tens of gigabytes per second within minutes. For a periodic real-time session, residency is the only budget-compatible choice.

What vLLM and SGLang provide, and what they do not. The state-of-the-art interactive engines provide exactly this residency. vLLM’s resumable-request API [vLLM Team, 2026] makes a session one long-lived request: append each frame’s chunk, reuse prior KV, no per-frame resubmission and no movement. SGLang’s streaming sessions do the same — each chunk is appended into a persistent GPU sequence — a feature Thinking Machines Lab built and upstreamed to serve its interaction models [Zheng et al., 2024, Thinking Machines Lab, 2026]. This is the correct serving primitive for a session — vLLM-realtime is our baseline throughout — but it *bounds* nothing: the resident context grows toward `max_model_len` for as long as the session lives, and neither engine offers a notion of per-frame schedulability or admission. Unbounded state taxes whichever resource it is parked in — FLOPs, PCIe, or HBM; residency selects HBM, where the tax accrues silently until the block pool is gone (§3). Metronome adds the missing half: bound the resident state.

3 Anatomy of the Collapse

We first dissect how unbounded resident-KV serving fails; everything else in the paper builds on this diagnosis. Unless noted, the workload is Qwen3-Omni-30B-A3B (FP8) [Qwen Team, 2025] on one Blackwell GPU behind the real client/gateway/worker stack, with a 2 s frame budget and N distinct, phase-staggered real-audio streams (full setup in Section 5.1).

The failure is a memory cliff, not a compute drift. Over a 90 s burst the unbounded baseline is flawless: latency stays flat far past a hundred concurrent sessions, because every session’s resident context is still small (Figure 4a). Run the *same* load for five minutes and the shape changes entirely (Figure 4b): latency holds at a few milliseconds and then jumps *discontinuously* to a wall. That binary shape is the tell that the trigger is not attention compute but memory.

The mechanism is simple. Each frame appends to every session’s resident KV, so N sessions steadily consume the engine’s fixed block pool; when the pool saturates, the scheduler can no longer allocate blocks and moves every running session to the waiting queue. An in-engine stat logger catches this in the act (Figure 5): pool occupancy climbs monotonically to capacity, at which instant the running count drops to zero and all N sessions queue. The stall never recovers, because audio keeps arriving open-loop and the stalled sessions can never catch up.

The collapse is metastable. Whether a given run collapses is a race between two clocks: the time the pool takes to fill and the length of the session. At the concurrencies of Figure 4b the two are comparable, so run-to-run variance in fill rate — output length, prefix-cache sharing across streams — decides the winner: identical five-minute runs collapse in 4/10 cases, while windowed KV never tips. A replication of the same twenty-run matrix in seeded random order on a different day walled in *every* vanilla run (10/10, against 0/10 windowed): the tip

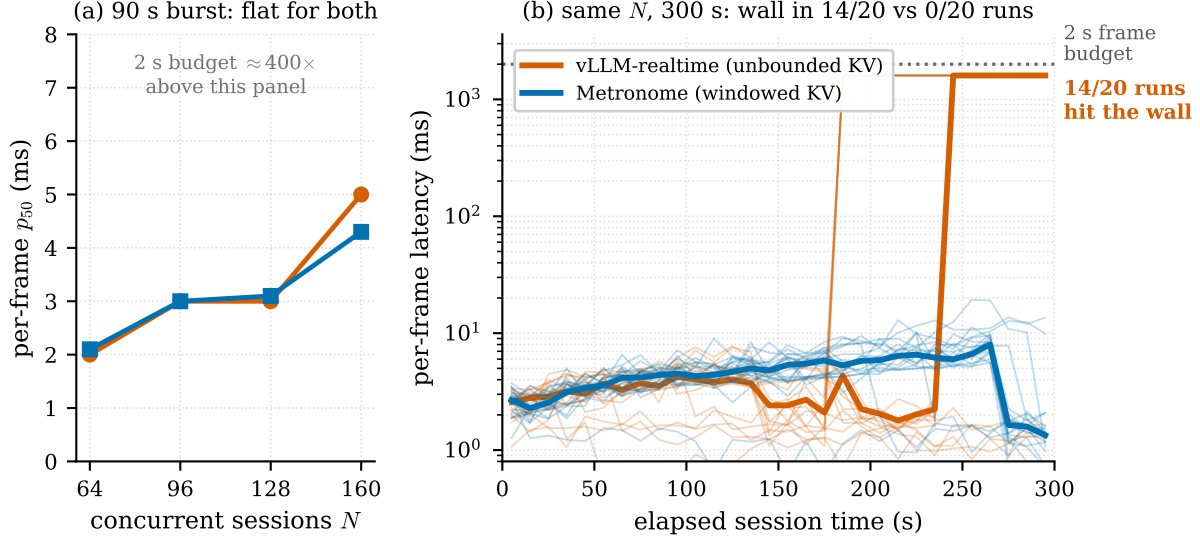


Figure 4. Bursts are fine; minutes are a gamble. (a) Fresh single- N runs over 90 s: per-frame latency is flat for *both* policies to $N=160$, hundreds of times under the 2 s budget — there is no short-burst capacity problem. (b) The *same* concurrencies over 300 s (log scale, 10 s bucket medians; both twenty-run batches pooled — ten fresh runs per policy at each $N \in \{96, 128\}$; thin lines individual runs, bold the median). With unbounded resident KV, the block pool saturates, the scheduler stalls, and latency jumps in one step to the ~ 1.6 s wall in 14/20 runs (4/10 on the fixed-order day, 10/10 in the seeded randomized-order replication); the others end before their pool fills. Windowed KV stays flat in every run (0/20). The wall is reached by *duration*, not concurrency — a 90 s test cannot see it. Per-run detail in Appendix D.

rate itself moves with the day’s fill rate, exactly as a race should, while the asymmetry is the invariant. The randomness is not inherent: a model that fills its pool much faster than the session ends collapses *deterministically* — MiniCPM-o-4.5 reproduces the wall this way (§5.5) — and Section 3.1 makes the race quantitative.

The collapse is silent. The stalled engine does not miss its deadlines — it returns *empty* frames on time. The worker caps how long a tick waits for a session’s tokens before returning (at $0.8B$, which is why the wall in Figure 4b sits at ~ 1.6 s rather than at the budget), so stalled ticks still return under budget and the deadline-miss counter reads zero throughout the collapse. Per-frame latency, the metric an operator would alarm on, is green until the step. What the user experiences is not a late frame but a conversation that goes quiet mid-call: a monitoring stack watching latency and deadline misses — the natural dashboards for this workload — sees a healthy system stop talking.

3.1 The cliff is predictable

The stat-logger traces reveal more than the trigger: they show the collapse obeys a first-order model. Pool occupancy under unbounded KV rises *linearly* — each of N sessions appends KV at a steady per-session rate r (fraction of pool per second), so

$$\rho(t) = \rho_0 + Nr t, \quad t_{\text{sat}} = \frac{1 - \rho_0}{Nr}, \quad (1)$$

and a run collapses iff t_{sat} is shorter than the session. Fitting r on the early trace predicts the measured saturation instant within a few percent on the 30B and within $\sim 13\%$ on MiniCPM-o

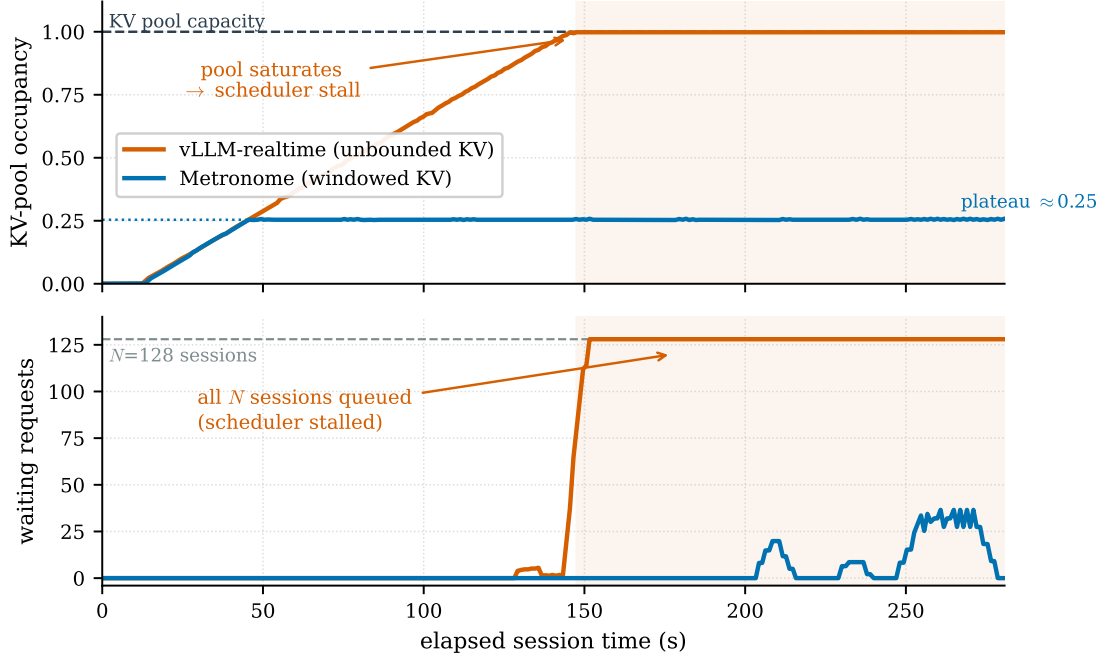


Figure 5. The trigger, caught in the act ($N=128$, 300 s, in-engine stat logger). **Top:** KV-pool occupancy. Unbounded resident KV climbs monotonically to pool capacity; in-engine windowed KV frees blocks behind the window and *plateaus* at roughly a quarter of the pool. **Bottom:** at the instant of saturation the scheduler can no longer allocate blocks and moves all N sessions to the waiting queue (shaded), a hard stall that never recovers under open-loop audio. The windowed policy sees only transient, self-healing preemption late in the run and never approaches saturation.

(Figure 6a). The metastability of Section 3 now has numbers: where t_{sat} is comparable to the session length, ordinary variance in r moves runs across the boundary, while a faster-filling configuration sits far below it and collapses every time.

Bounding the resident context replaces the linear ramp with a plateau. Each windowed session retains at most a fixed share of the pool, so occupancy settles at $\rho_{\infty} = \rho_0 + Ns(W)$, linear in N with slope $s(W)$ — about 0.2% of the pool per session at our operating window (Figure 6b). Memory turns from a hidden failure clock into a provisionable budget: the plateau extrapolates to an absolute ceiling of roughly 500 resident sessions, more than double the deadline-schedulable concurrency the admission controller discovers (§5.3). Bounded-KV serving is therefore *compute*-limited: the deadline binds long before memory. This is the exact reverse of the vanilla failure, where memory kills sessions whose compute the GPU could easily carry.

4 Metronome: Bound the State

Metronome is one organizing move — bound each session’s resident KV — and two mechanisms that follow from it. The *in-engine windowed KV* does the bounding and removes the cliff (§4.1); the *deadline-aware admission controller* acts on the latency signal the window makes trustworthy (§4.2). Figure 2 shows where each sits. The end-to-end path is deliberately conventional: WebSocket clients stream 20 ms audio to a Go gateway, which once per tick issues a single batched Step over gRPC for all due sessions; a GPU worker owns the engine, feeds

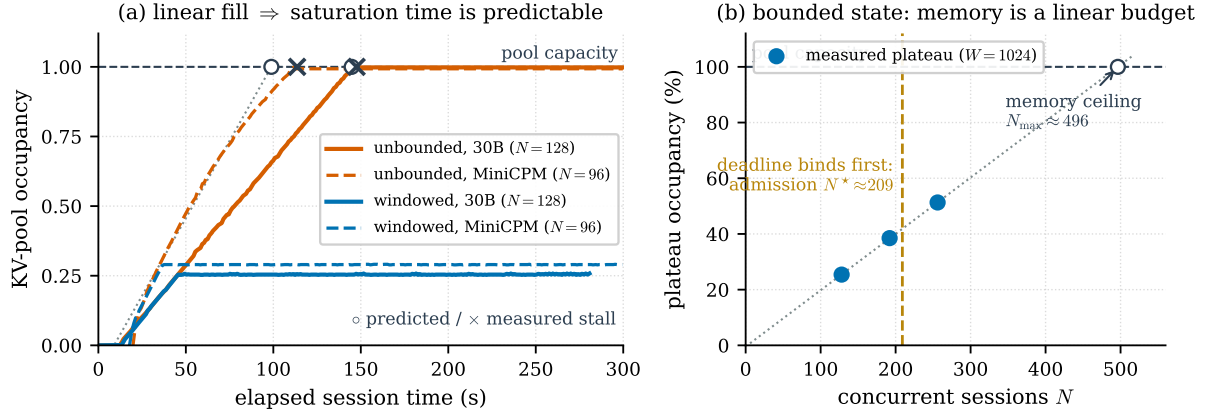


Figure 6. The collapse obeys a first-order model. (a) Unbounded pool occupancy rises linearly (Eq. 1); a straight-line fit on the early trace (dotted) predicts the measured stall (×) within a few percent on Qwen3-Omni-30B (145 s predicted vs. 148 s) and within $\sim 13\%$ on MiniCPM-o (99 s vs. 114 s). Windowed occupancy plateaus far below capacity. (b) The windowed plateau is linear in N ($\sim 0.2\%$ of pool per session, $W=1024$), extrapolating to a memory ceiling of ≈ 500 sessions — well above the deadline-schedulable $N^* \approx 209$ that admission discovers, so with bounded state the deadline binds before memory.

each session’s long-lived resumable request, and returns the tokens produced since the last tick. Identical clients and gateway drive every configuration we evaluate.

4.1 In-engine windowed KV, anchored by sinks

Metronome bounds each session’s resident state to a fixed-shape set: the most recent W tokens plus the session’s first few tokens, kept pinned. The *window* bounds both quantities that grow with session age — per-frame attention becomes $O(W)$ and resident KV is capped — and the pinned *sink* tokens (one–two KV blocks) anchor generation quality: full-attention backbones concentrate softmax mass on the earliest positions [Xiao et al., 2024], so a bound that evicted them would corrupt free-running decode. Section 5.4 validates each half by ablation. Per-token KV-reduction techniques [Shazeer, 2019, Ainslie et al., 2023, Zhang et al., 2023, Liu et al., 2023, Li et al., 2024, Liu et al., 2024] shrink what each token stores; we bound *which* tokens stay resident, and the two compose.

The realization is minimal. vLLM already implements sliding-window attention, including freeing KV blocks behind the window, but only for models that *declare* a window — and interaction models do not, so the path lies dormant. Metronome activates it and completes it: W is installed on the decoder-attention layers of the omni text backbone at construction time, the KV manager pins each request’s first blocks instead of freeing them, and the attention kernel’s windowed mask admits $[0, S) \cup [t - W, t]$ (implementation detail in Appendix A). No request recycling, no re-encode, no application-level machinery: the resident request keeps growing logically while the engine attends and retains only within the bound. The obvious alternative — recycling the resumable request at window boundaries in the application — achieves the same memory horizon at a re-encode cost the in-engine bound never pays (§5.2).

4.2 Deadline-aware admission on a faithful signal

Beyond the schedulable concurrency N^* , admitting one more session degrades *every* session, because continuous batching shares the GPU across all of them; the right behavior is to admit

up to N^* and shed the rest. But N^* depends on model, window, hardware, and arrival pattern, so a hand-set cap is brittle — it must be *discovered* from feedback. Metronome’s controller is online and model-free: an AIMD loop [Chiu and Jain, 1989, Jacobson, 1988] on the measured per-frame latency, which lowers the admission cap multiplicatively when latency nears a target fraction of the budget and raises it additively to probe when there is headroom. Arrivals beyond the cap receive a clean overload rejection rather than degraded service.

This design makes a falsifiable claim: the loop works *only if* latency is a monotone signal of load. On bounded KV it is, and the controller should converge on N^* . On unbounded KV the signal is flat until the pool exhausts (§3), so the same controller should see nothing but headroom and over-admit into the wall. Section 5.3 runs both arms.

5 Evaluation

We ask four questions, one per subsection: does bounding the state remove the cliff (§5.2)? Does admission converge only on the bounded signal (§5.3)? Does the bound cost answer quality (§5.4)? And does the failure generalize across models (§5.5)?

5.1 Setup

All experiments run end-to-end — WebSocket clients, Go gateway, GPU worker — on one NVIDIA RTX PRO 6000 Blackwell, with real audio (LibriSpeech and spoken-question clips) streamed continuously in 20 ms chunks; each session is a distinct, phase-staggered stream, so prefix-cache deduplication cannot inflate capacity. We report per-frame latency percentiles bucketed by elapsed time, frame-delivery cadence, and answer correctness under load. Two methodology notes: every data point runs on a freshly started worker, because a long-lived worker silently inflates apparent degradation (Appendix C); and running these models on Blackwell required four engine bug-fixes, which ship with our artifact (Appendix B). The baseline everywhere is unmodified vLLM-realtime resumable-KV serving — the strongest available starting point, and the same engine, stack, and load as Metronome in every comparison.

5.2 Bounding the state removes the cliff

Headline: 14/20 walls become 0/20. With identical stack, model, and load, differing only in Metronome’s bounded KV, unbounded serving walls in 14/20 runs pooled across the two twenty-run batches; windowed serving in 0/20 (Figures 4b and 5; per-run detail in Appendix D). The insurance is free: in runs where the baseline happens not to collapse, the two policies sit within a few milliseconds of each other.

In-engine beats application-level. Recycling the resumable request at window boundaries — the same memory horizon, implemented above the engine — also avoids the wall, but its periodic re-encode grows over the call, while the in-engine window holds flat (Appendix D).

The window needs no tuning. Latency is unchanged across window sizes up to ~ 80 s of context (Appendix D), and the memory ceiling of §3.1 sits more than double the schedulable concurrency. With wide margins on both sides, pick a window for quality and let admission handle the deadline.

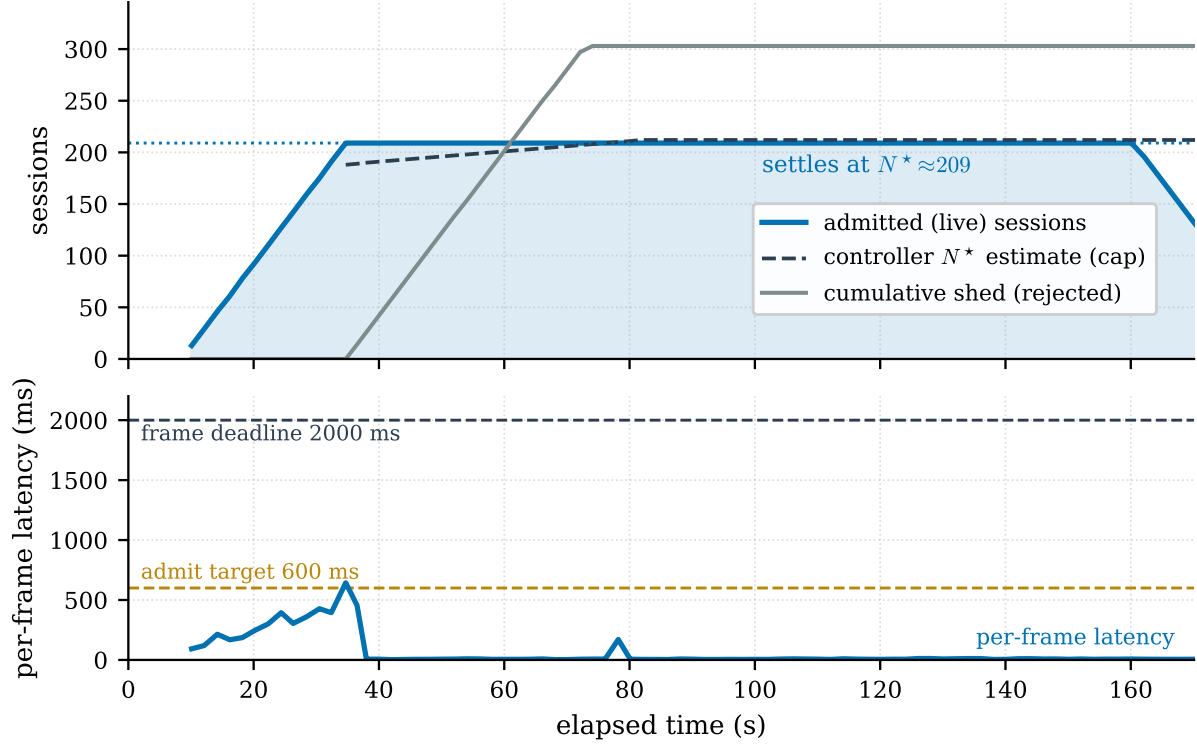


Figure 7. Online admission discovers the schedulable concurrency — on the bounded-KV signal (open-system ramp: 512 sessions offered at 8/s, 600 ms latency target, windowed worker). **Top:** admitted concurrency and the controller’s cap rise with arrivals and settle at $N^* \approx 209$; the surplus is shed cleanly (cumulative count, same axis). **Bottom:** per-frame latency — the only signal the controller uses — probes the target once during ramp-up, then holds far under the 2 s deadline (steady p_{99} of 12 ms). Against unbounded KV the identical controller admits past this point on a flat signal and ends pinned at the ~ 1.6 s wall (§5.3).

5.3 Admission converges only with bounded state

With the bound, the controller converges. In an open-system overload against the windowed worker, the cap and the admitted count climb together as sessions arrive; when per-frame latency first touches the target, the controller caps, and the system settles at $N^* \approx 209$ — discovered online, with no hand-set capacity number anywhere in the configuration (Figure 7). Latency holds far under the deadline for the rest of the run; the surplus is shed cleanly.

Without it, the identical controller over-admits into the wall. Against unbounded KV the latency signal stays flat as sessions pour in, so the controller reads pure headroom and admits far past the limit the windowed system identified, while the admitted sessions’ resident KV silently fills the pool. The run ends pinned at the wall: shedding late cannot rescue sessions that are already resident. The dependency runs in both directions — bounded state makes the latency signal faithful, and a faithful signal makes admission converge.

5.4 Quality: both halves of the bound, validated by ablation

Bounding a full-attention model’s context could harm answers, so we ablate each half of the bound in the two decoding regimes an interaction model runs.

Turn-based decoding: the window alone is quality-free. In the regime a spoken-QA deployment runs — each response an EOS-terminated turn over recent context — windowed and unbounded serving are indistinguishable: under load, every session states the correct answer to its spoken question, with per-frame correctness statistically identical. Each turn’s context fits inside the window; the sinks are not even exercised.

Free-running decode: the sinks are load-bearing. The harder regime decodes continuously on one resident request. We play each session a sequence of distinct spoken questions for five minutes and track whether it answers the one currently playing (Figure 8). Ablating the sinks makes every windowed variant decay toward zero once the window slides past the session start, while the unbounded baseline holds steady; engine metrics stay clean throughout, so the decay is the model-side attention-sink effect [Xiao et al., 2024], not a serving artifact. Restoring the sinks removes it: the full bound holds an age-independent profile at or above the baseline and answers a fresh synthesized-voice question played late in the call. A zero-sink control on the identical kernel reproduces the decay, pinning the recovery on the sinks; their cost is two KV blocks per session and unchanged latency.

What sinks do not buy: memory. Recall beyond the horizon is impossible under any fixed bound. Asked late for the session’s *first* question, a session confidently names the most recent one it can still see — a coherent wrong answer, which is what the lenient keyword scorer credits as the full bound’s nonzero recall score in Figure 8b. Recall is a task for retrieval, not residency.

Sizing the bound. A sweep with boundary controls (Table 2) yields one rule: *pin structure, not content, and keep the window generous*. $S=16$ — one KV block, the chat header — works best, and quality falls as the pin reaches into the first turn’s content: pinned content stays semantically live (sessions keep answering the pinned first question minutes later), and pinning the model’s own first output tokens collapses generation into template echo. The window side is forgiving — $W=2048$ matches $W=1024$ — but sinks cannot rescue a window too small for the task ($W=512$).

5.5 Generality across four models

Capacity is architectural; the cliff is not. Fresh single- N capacity spans an order of magnitude across the four models (Figure 9): Qwen2.5-Omni-7B is bound by its heavyweight audio encoder, while MiniCPM-o-4.5 and the MoE 30B sustain far more sessions on the same GPU. The cliff, by contrast, travels with the serving path.

On MiniCPM-o, the coin flip becomes certainty. A dense backbone with 1 s frames fills its pool in under two minutes — a fifth of the ten-minute session, far below the metastable boundary the 30B sat near — so, exactly as the race of §3.1 predicts, the run hard-stalls on schedule, deadline-miss counter at zero throughout. Windowed KV under the same load holds single-digit-millisecond medians for the full ten minutes. Every model on this path has the same KV growth; the two we drove to the wall differ only in how fast they reach it.

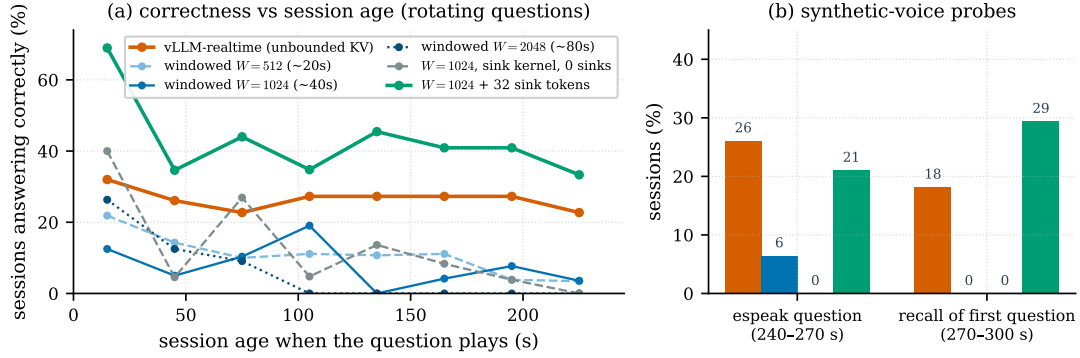


Figure 8. The bound needs both halves: ablating the sinks fails free-running generation (continuous forced decoding, $N=32$, 300 s, Qwen3-Omni-30B; rotating spoken questions, one per 30 s segment). **(a)** Sessions answering the currently playing question, by session age: unbounded KV holds a steady low plateau (continuous greedy decoding is degenerate for all policies); every sink-ablated window declines toward zero as it slides past the session start — including the sink-capable kernel run with the sinks off (grey control) — while the full bound, $W=1024$ plus 32 pinned sink tokens (green), holds an age-independent profile at or above the baseline. **(b)** A synthesized-voice comprehension check late in the session and a recall probe about its start: the full bound answers the fresh question; its nonzero recall *score* is the lenient scorer crediting coherent in-window answers, not beyond-horizon memory (§5.4). Turn-based decoding shows none of this.

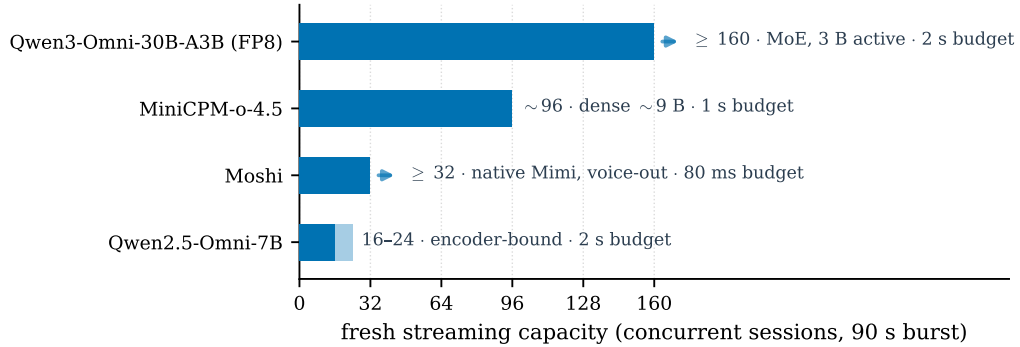


Figure 9. Fresh single- N streaming capacity across four interaction models (90 s burst, one freshly started worker per point). A $\geq$ arrow means the run was still flat at the largest N tested, so the value is a lower bound. Short-burst capacity is set by architecture (audio-encoder weight, MoE sparsity, frame budget); the long-duration cliff of §3 is a property of the serving path and reproduces across models.

6 Discussion: Beyond Voice

Nothing in the failure mechanism is specific to audio. Any serving loop that holds *unbounded per-session state* against a *recurring deadline* manufactures the same cliff: an agent whose scratchpad and tool-call history grow with every step of a long-running task, a streaming-video assistant accumulating frame context, a stateful RAG cache that grows per interaction. In each case the state grows monotonically, is pinned while the session lives, and exhausts a fixed pool on a clock that per-request latency does not see. The leading indicator is state occupancy, and the principled fix is the same: bound the resident state per session, and size the bound to the task.

The engine-design implication is that a per-session state bound should be a first-class serving parameter, not an emergent property of `max_model_len`. Today the model-length cap

is the only backstop on a resident session, and it is a crash boundary rather than a policy: a streaming request that reaches it does not degrade or stop cleanly — in our engine it kills every co-resident session at once (Appendix B). An engine that exposes “retain each session’s first S and last W tokens, freeing everything between” as an API — exactly the sink-anchored bound validated in §5.4 — would give every real-time deployment the slope that control depends on, without patching model definitions as we do.

7 Limitations and Future Work

Scope of hardware and engines. All results are from one Blackwell GPU and one engine (vLLM-realtime); the wall is demonstrated on two models (Qwen3-Omni-30B, MiniCPM-o-4.5), admission and quality on the 30B. A second engine with streaming sessions (e.g. SGLang, blocked here by a CUDA toolchain conflict) would strengthen generality.

Statistical coverage. The wall statistics are the two twenty-run batches of §5.2; because the tip rate is regime-dependent (§3), we report the asymmetry rather than a calibrated probability. Admission, per-model capacity, and the MiniCPM-o wall results are single fresh runs.

Bound-implementation scope. The full sink-anchored mask lives in the engine’s Triton attention kernel (FlashAttention’s window primitive cannot express $[0, S) \cup [t - W, t]$); its quality effect is measured on the 30B at $N=32$, across the free-running conditions of Table 2. The serving-side experiments run the window half alone on the FlashAttention path — equivalent for everything they measure, since the sinks add one or two resident blocks per session and leave latency flat. Replicating the quality boundary on a second backbone remains open.

Workload and modality. We serve continuous full-duplex streams of real audio; richer conversational dynamics (turn-taking, barge-in) are out of scope by design, and the omni models emit text from the thinker — only Moshi produces streamed voice.

8 Related Work

LLM serving engines. Modern engines optimize aggregate throughput for ephemeral requests via continuous batching [Yu et al., 2022, Kwon et al., 2023], paged KV [Kwon et al., 2023], high-throughput single-GPU execution [Zheng et al., 2024, Sheng et al., 2023], and multiplexing across models [Li et al., 2023]; a second line schedules or splits prefill and decode [Agrawal et al., 2024, Zhong et al., 2024, Patel et al., 2024]. All target ephemeral-request goodput and are deadline-agnostic, and their state-movement machinery — reclamation, swapping, offload [Kwon et al., 2023, Sheng et al., 2023] — presumes idle gaps that a periodic session never has (§2). Metronome treats the resident-KV mechanism these engines now expose [vLLM Team, 2026] as its substrate and adds the state bound and deadline-aware control they lack.

Attention, long context, and KV reduction. IO-aware exact attention kernels [Vaswani et al., 2017, Dao et al., 2022, Dao, 2024, Ye et al., 2025] make per-frame attention fast but still scale with context; sparse and streaming attention bound the cost structurally [Dai et al., 2019, Child et al., 2019, Zaheer et al., 2020, Beltagy et al., 2020, Jiang et al., 2023, Xiao et al., 2024]; per-token KV-cache reduction [Shazeer, 2019, Ainslie et al., 2023, Zhang et al., 2023, Liu et al., 2023, Li et al., 2024, Liu et al., 2024] is orthogonal, shrinking each token’s footprint where our window bounds how many tokens stay resident (§4.1). We repurpose the engine’s sliding-window

machinery for the *serving* problem — bounding resident memory and per-frame compute on a live streaming session — and quantify it under a frame deadline, including the sink anchoring that keeps it quality-free.

Real-time scheduling and admission. Recurring deadlines and admission control are classical [Liu and Layland, 1973], AIMD is the canonical feedback controller [Chiu and Jain, 1989, Jacobson, 1988], and SLO-aware DNN serving enforces latency targets through predictability and adaptive batching [Gujarati et al., 2020, Crankshaw et al., 2017]. We instantiate deadline-aware admission online, over a measured per-frame latency signal, and show the signal itself must be earned by bounding state.

Interaction and speech models. Full-duplex and streaming speech dialogue [Défossez et al., 2024, OpenBMB, 2026, Xie and Wu, 2024, Fang et al., 2024], omni understanding [Xu et al., 2025, Chu et al., 2024, Tang et al., 2024], frontier interaction models [Thinking Machines Lab, 2026], and the encoders and codecs beneath them [Radford et al., 2023, Borsos et al., 2022, Wang et al., 2023, Défossez et al., 2022] define the workload. Our contribution is at the serving layer, complementary to the models.

9 Conclusion

Real-time interaction sessions are periodic real-time tasks with unbounded, pinned per-session state, and that combination fails in a way dashboards cannot see: not by slowing down, but by a memory-triggered, metastable, silent cliff. One move fixes more than it appears to. **Bounding each session’s resident state turns the cliff into a slope:** it keeps every session on beat, makes the collapse predictable instead of a coin flip, and — because graceful degradation is what makes a metric trustworthy — turns per-frame latency into a signal precise enough to admit exactly to the deadline. Metronome realizes the principle minimally: a sink-anchored in-engine KV window and the admission controller it enables, measured end-to-end on real audio across four models, with both halves of the bound validated by ablation rather than assumed. The lesson generalizes past voice: wherever a real-time loop accumulates unbounded per-session state, a cliff is being manufactured, and bounding that state is how one earns back the slope that control depends on.

Acknowledgements

This paper was produced using Pine Copilot’s voice-directed *whisper coding* workflow [Pine AI, 2026], in which the authors specify, discuss, and review the work by voice while a coding agent, Claude Code with Claude Opus 4.8 and Claude Fable 5, carries out the planning, coding, experiments, and paper writing. The authors thank BSQL Networking for hosting the NVIDIA RTX PRO 6000 GPU.

References

Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency trade-

- off in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- Zalán Borsos, Raphaël Marinier, Damien Vincent, Eugene Kharitonov, Olivier Pietquin, et al. AudioLM: a language modeling approach to audio generation. *arXiv preprint arXiv:2209.03143*, 2022.
- Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
- Dah-Ming Chiu and Raj Jain. Analysis of the increase and decrease algorithms for congestion avoidance in computer networks. *Computer Networks and ISDN Systems*, 17(1):1–14, 1989.
- Yunfei Chu et al. Qwen2-Audio technical report. *arXiv preprint arXiv:2407.10759*, 2024.
- Daniel Crankshaw, Xin Wang, Giulio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. Clipper: A low-latency online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2017.
- Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.
- Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Alexandre Défossez, Jade Copet, Gabriel Synnaeve, and Yossi Adi. High fidelity neural audio compression. *arXiv preprint arXiv:2210.13438*, 2022.
- Alexandre Défossez, Laurent Mazaré, Manu Orsini, et al. Moshi: a speech-text foundation model for real-time dialogue. *arXiv preprint arXiv:2410.00037*, 2024.
- Qingkai Fang, Shoutao Guo, Yan Zhou, Zhengrui Ma, Shaolei Zhang, and Yang Feng. LLaMA-Omni: Seamless speech interaction with large language models. *arXiv preprint arXiv:2409.06666*, 2024.
- Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. Serving DNNs like Clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.

- Van Jacobson. Congestion avoidance and control. In *Symposium Proceedings on Communications Architectures and Protocols (SIGCOMM)*, 1988.
- Albert Q. Jiang et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, et al. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, et al. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2023.
- C. L. Liu and James W. Layland. Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM*, 20(1):46–61, 1973.
- Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, et al. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, et al. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning (ICML)*, 2024.
- OpenBMB. MiniCPM-o 4.5: Towards real-time full-duplex omni-modal interaction. *arXiv preprint arXiv:2604.27393*, 2026.
- Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024.
- Pine AI. Pine AI: The most natural human-computer interface is your voice. Blog post, 2026. URL <https://www.19pine.ai/blog/pine-ai-the-most-natural-human-computer-interface-is-your-voice>. Accessed 2026-07-02.
- Qwen Team. Qwen3-Omni technical report. *arXiv preprint arXiv:2509.17765*, 2025.
- Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *International Conference on Machine Learning (ICML)*, 2023.
- Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.
- Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, et al. FlexGen: High-throughput generative inference of large language models with a single GPU. In *International Conference on Machine Learning (ICML)*, 2023.

- Changli Tang, Wenyi Yu, Guangzhi Sun, Xianzhao Chen, Tian Tan, et al. SALMONN: Towards generic hearing abilities for large language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- Thinking Machines Lab. Interaction models: A scalable approach to human-AI collaboration. <https://thinkingmachines.ai/blog/interaction-models/>, 2026. Blog post, May 11, 2026.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- vLLM Team. vLLM v0.23: Resumable requests and the realtime API. <https://docs.vllm.ai/>, 2026. Release documentation.
- Chengyi Wang, Sanyuan Chen, Yu Wu, Ziqiang Zhang, Long Zhou, et al. Neural codec language models are zero-shot text to speech synthesizers. *arXiv preprint arXiv:2301.02111*, 2023.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.
- Zhifei Xie and Changqiao Wu. Mini-Omni: Language models can hear, talk while thinking in streaming. *arXiv preprint arXiv:2408.16725*, 2024.
- Jin Xu et al. Qwen2.5-Omni technical report. *arXiv preprint arXiv:2503.20215*, 2025.
- Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, et al. FlashInfer: Efficient and customizable attention engine for LLM inference serving. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025.
- Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.
- Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, et al. Big Bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, et al. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, et al. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

A In-Engine Windowed KV: Implementation

Window half. vLLM supports sliding-window attention for models that declare it: when a decoder layer carries a `sliding_window` attribute, the engine constructs a `SlidingWindowSpec` for that layer’s KV cache and the FlashAttention backend [Dao et al., 2022, Dao, 2024, Ye et al., 2025] applies a windowed causal mask. The KV manager then frees blocks that fall entirely behind the window, which is what bounds resident memory — the property the collapse analysis of Section 3 turns on. Metronome sets `sliding_window=W` at model-construction time on the shared decoder-attention class that the omni text backbones build, so every decoder layer reports a windowed spec; no scheduler, allocator, or API change is required.

Sink half. FlashAttention’s window primitive cannot express the sink-anchored mask $[0, S) \cup [t-W, t]$, so the sink half routes the decoder layers to vLLM’s Triton unified-attention backend and extends its kernel: the windowed mask gains the OR-term $k < S$, and the tile loop gains $\lceil S/\text{tile} \rceil$ extra leading iterations remapped onto the sink tiles — the online softmax is order-invariant, so accumulating them first is exact, and in segmented-decode mode only segment 0 takes them, merged exactly once. On the memory side, the sliding-window KV manager pins each request’s first $\lceil S/\text{block} \rceil$ blocks instead of freeing them, and the per-request admission cap counts the pinned blocks so pool sizing stays exact. The kernel is verified against a float32 reference of the union mask (mixed prefill/decode, grouped-query attention, both launch modes), with freed blocks NaN-poisoned to prove nothing outside sinks $\cup$ window is ever read; with $S=0$ every touched path is byte-identical to stock. The test ships with the artifact.

B Enabling Omni Streaming on Blackwell

Running the omni models on vLLM 0.23 (the first release with resumable requests) on an SM120 Blackwell GPU required four engine bug-fixes, without which the models do not initialize or stream; the combined patch that ships with the artifact adds a fifth change, the in-engine bounded KV itself — the sliding window and its sink retention (Appendix A).

1. **cu_seqlens device.** The shared multimodal encoder attention builds `cu_seqlens` on the feature tensor’s device (CPU during memory profiling) while `q/k/v` are on CUDA, crashing the varlen flash-attention op [Dao et al., 2022]. We coerce it to the query device.
2. **Capability gate.** FlashInfer [Ye et al., 2025] gates SM12.x support on the *toolkit* CUDA version. We gate on the *runtime* version so Blackwell is recognized when the local `nvcc` is older.
3. **JIT toolchain.** FlashInfer’s bundled CCCL headers reject the CUDA-13.2 `nvcc`; the documented escape macro (`CCCL_DISABLE_CTK_COMPATIBILITY_CHECK`) unblocks the JIT across a CUDA minor version.
4. **mRoPE off-by-one.** A Qwen3-Omni position-id routine double-counts the audio start token, overshooting the sequence length and crashing precisely when an audio block ends a streamed chunk — the per-frame append case. A one-line reconcile fixes it and is a no-op for every previously working input.

With these, Qwen3-Omni-30B, Qwen2.5-Omni-7B, and MiniCPM-o-4.5 all load and stream; Moshi runs on its own native stack. The evaluation also surfaced a sixth hazard: a resident

streaming request that reaches `max_model_len` does not stop cleanly — the engine core crashes on a shape mismatch, killing *every* co-resident session at once. Unbounded per-session state is dangerous at more than one boundary; our long-horizon runs size `max_model_len` above the session’s growth to avoid it, and Section 6 argues the general fix is a first-class per-session state bound.

C Measurement Methodology: Fresh Worker Per Point

Capacity curves for this workload must be measured with a freshly started worker per data point. Measuring a capacity curve as a *sweep* of many N values on a single long-lived worker silently inflates the apparent degradation at the larger N values, which arrive later in the sweep: residual engine state accumulates across points, and slowly varying ambient conditions (in our case, intermittent contention from an unrelated GPU job) fold into whichever points run last. In our data this made the unbounded baseline appear to collapse at $N=128$ – 160 in a 90 s sweep, when a fresh worker per point is flat to $N=160$ (Figure 4a), and it manufactured an apparent capacity cliff for Moshi at $N=24$ that likewise vanished under fresh measurement. Every number in this paper is therefore measured with one fresh worker per point. We recommend this as standard practice for interaction-serving capacity measurement: the workload’s long-lived resident state makes it unusually sensitive to measurement-harness state.

D Additional Results

Per-run wall outcomes. Table 1 details the per-run outcomes behind Figure 4b and §5.2: the fixed-order batch and the seeded randomized-order replication (the shuffle and per-run log ship with the artifact).

Table 1. The latency cliff, per operating point (5 fresh runs each, 300 s; bucket p_{50}). Unbounded resident KV is a metastable gamble whose tip rate moves with the day’s fill rate (§3.1); in-engine windowed KV is flat in every run of both batches.

	vanilla (unbounded KV)	in-engine windowed KV
batch 1 (fixed order), $N=96$	wall in 2/5 runs ($\rightarrow \sim 1.6$ s), else ~ 2 – 5 ms	~ 1 – 2 ms, 0/5 wall
batch 1 (fixed order), $N=128$	wall in 2/5 runs ($\rightarrow \sim 1.6$ s), else ~ 2 – 5 ms	~ 2 – 12 ms, 0/5 wall
batch 2 (randomized), $N=96$	wall in 5/5 runs ($\rightarrow \sim 1.6$ s)	flat few ms, 0/5 wall
batch 2 (randomized), $N=128$	wall in 5/5 runs ($\rightarrow \sim 1.6$ s)	flat few ms, 0/5 wall

In-engine windowing vs. application-level recycling. Figure 10 details the comparison summarized in §5.2: three policies with the same memory horizon.

Window-size ablation. Figure 11 sweeps the window size at fixed load. Latency is flat up to a 2048-token (~ 80 s) window; the lone p_{90} uptick at 4096 tokens carries a p_{99} comparable to the smaller windows’, so we attribute it to single-run tail noise rather than a clean attention effect. A memory horizon well beyond the ~ 30 s most interactions need is free.

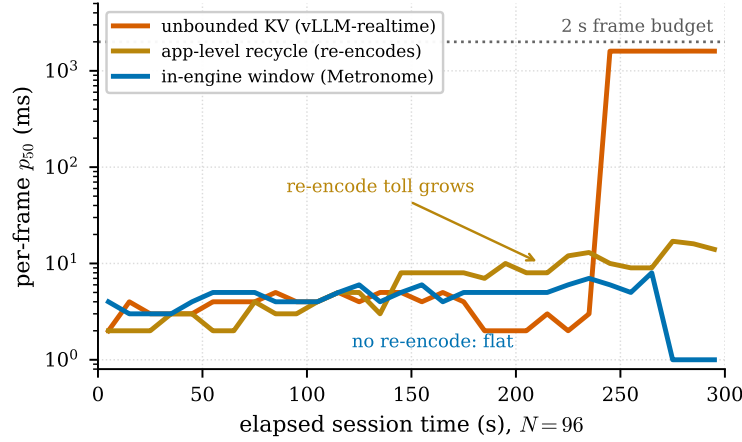


Figure 10. In-engine windowing beats application-level recycling. Three policies with the same memory horizon at $N=96$ over 300 s (per-frame p_{50} , 10 s buckets). Unbounded resident KV hits the wall at ~ 240 s. The application-level recycling proxy avoids the wall but its periodic re-encode grows over the call (p_{50} to ~ 14 – 17 ms, p_{90} to 36 ms). The in-engine window bounds the horizon without ever re-encoding.

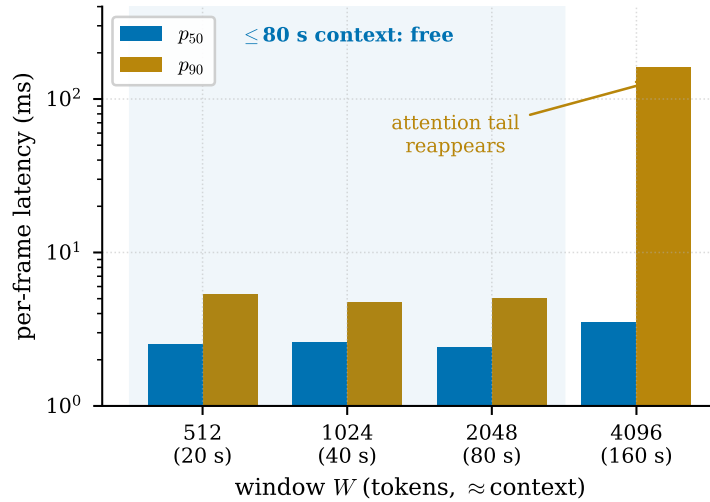


Figure 11. A generous window is free. Window-size ablation (in-engine windowed KV, $N=128$, 60 s runs): p_{50}/p_{90} stay ~ 5 ms up to a 2048-token (~ 80 s) window.

Turn-based quality detail. In the turn-based probe of Section 5.4 ($N=96$, 75 s sessions, EOS-terminated turns), answer-stated counts are 96/96 sessions for both vanilla and windowed policies, with per-frame correctness $\sim 70\%$ vs. $\sim 68\%$ (statistically indistinguishable). The gap from 100% per-frame is genuine model behavior on looped, sometimes partial audio and is identical across policies.

Free-running conditions: sweep and boundary controls. Table 2 collects every free-running condition — the ablation of Figure 8 plus the sink/window sweep, with boundary controls at the exact token-layout edges. The controls localize the harm of over-pinning (§5.4): a pin that *truncates* the first audio block is harmless ($S=32$), refuting the malformed-block hypothesis, and the failure signatures in the outcome column track how much first-turn *content* the pin

keeps semantically live. Engine-side, the rows differ only as block arithmetic dictates (pool column: plateau $\propto W$, plus $\sim 0.1\%$ of the pool per pinned sink block) and latency is flat in every run, so the quality differences are entirely model behavior. The unbounded baseline’s pool was still climbing when its 300 s session ended — at $N=32$ the session outlives the run, the benign side of the two-clock race of §3.1.

Table 2. All free-running probe conditions (Figure 8 and the sink/window sweep; $N=32$, 300 s, one fresh worker per row, per-frame $p_{99} \leq 3$ ms and zero errors throughout). *Mid-call*: sessions answering the currently playing question over ages 30–240 s. *Fresh Q*: the synthesized-voice question at 240–270 s. *Pool*: KV-pool occupancy plateau. Token layout: chat header [0, 14), first audio [14, 42), instruction [42, 53), assistant-open [53, 58), generated from 58.

(W, S)	pin covers	mid-call	fresh Q	pool	outcome
unbounded	—	23–27%	26%	74% $\uparrow$	steady; pool still filling at 300 s
(512, 0)	— (sinks ablated)	$\rightarrow 3\%$	7%	3.3%	decays as window slides
(1024, 0)	— (sinks ablated)	$\rightarrow 0\text{--}8\%$	6%	6.4%	decays
(2048, 0)	— (sinks ablated)	$\rightarrow 0\%$	0%	12.7%	decays
(1024, 0)	— (sink kernel, control)	$\rightarrow 0\%$	0%	6.5%	decays: kernel is not the cause
(1024, 16)	chat header	38–57%	45%	6.5%	recovers — best
(1024, 32)	+ $\sim 2/3$ of first audio	33–45%	21%	6.5%	recovers
(1024, 42)	+ complete first audio	20–32%	21%	6.6%	degraded: answers the pinned clip
(1024, 58)	+ instruction, assistant-open	25–37%	18%	6.8%	degraded: quotes the pinned clip
(1024, 64)	+ first 6 <i>generated</i> tokens	$\rightarrow 7\%$	8%	6.6%	collapses: template echo
(512, 32)	as (1024, 32)	$\rightarrow 0\%$	0%	3.5%	too-small window; sinks no rescue
(2048, 32)	as (1024, 32)	40–57%	42%	12.8%	recovers; matches $W=1024$